Lab 1: Verilog Instantiation

ECEN 127L: Tuesday 5:15 - 8:00 pm

Ryan Kiniris and Dylan Thornburg

Introduction

This lab demonstrates the basics and fundamentals of Verilog design by focusing on the creating and simulating custom characters made through a 7 segment display. By controlling the 7-segment display, using 8 custom alphabets that are mapped into 3-bit input codes. These characters are then used to create four-letter "words" on a simulated display, with the order of the letters controlled by a 2-bit formatting input. To get the 2-bit formatting we will implement a character generator and a 3-bit 4:1 multiplexer, as well as integrate it into them into a design through the given testbench. Overall, the lab highlights core digital design concepts, practicing Verilog instantiation, and gaining experience in simulation and debugging.

Procedure

After we compared our character maker modules, we created our verilog modules for our character makers and 4to1 mux. After that we added code within the test bench to hook up our modules as designed in our prelab schematics. We had to debug some errors (google docs not pasting correctly and reg/wire mistype), but then we compiled successfully. We initially tried to verify our design using the GUI, but we realized it was simpler to just read the printed debug statements Wolfe made for us. All code is in the appendix. Putting little snippets here would just muddy the procedure up.

Results

• When you first simulated your top-level design, did you find any problems? Were any of them internal to the character generators or were they mistakes in your connections at the top level?

No we actually didn't. We had some issues compiling, but simulating went smoothly.

• Were there problems that you needed help from the TA to resolve? Briefly explain what these were, and whether you feel you would be able to find similar types of problems in the future.

No, not really. We asked some questions about data types an syntax, and the answers from those questions helped us complete the lab, but we could've researched this online as well.

SIMULATION RESULTS

```
[dthornburg@linux402604 Lab1]$
[dthornburg@linux402604 Lab1]$ ./simv -no_save
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12-SP1-1_Full64; Runtime version V-2023.12-SP1-1_Full64;  Sep 30 17:49 2025
Start of the decoder test
sel = 0, charval0=0, charval1=1,charval2=2,charval3=3, sega=1111001,segb=1110111, segc=0111001, segd=0111000, @      0
sel = 1, charval0=0, charval1=1,charval2=2,charval3=3, sega=1110111,segb=1111001, segc=0111000, segd=0111001, @     10
sel = 2, charval0=0, charval1=1,charval2=2,charval3=3, sega=0011100,segb=1011100, segc=1111001, segd=1110111, @     20
sel = 3, charval0=0, charval1=1,charval2=2,charval3=3, sega=1011100,segb=0011100, segc=1110111, segd=1111001, @     30
sel = 0, charval0=2, charval1=1,charval2=2,charval3=1, sega=0011100,segb=1110111, segc=0111001, segd=1110111, @     40
sel = 1, charval0=2, charval1=1,charval2=2,charval3=1, sega=1110111,segb=0011100, segc=1110111, segd=0111001, @     50
sel = 2, charval0=2, charval1=1,charval2=2,charval3=1, sega=0011100,segb=1110111, segc=0111001, segd=1110111, @     60
sel = 3, charval0=2, charval1=1,charval2=2,charval3=1, sega=1110111,segb=0011100, segc=1110111, segd=0111001, @     70
sel = 0, charval0=6, charval1=1,charval2=0,charval3=7, sega=1010100,segb=1011100, segc=0111001, segd=0111110, @     80
sel = 1, charval0=6, charval1=1,charval2=0,charval3=7, sega=1011100,segb=1010100, segc=0111110, segd=0111001, @     90
sel = 2, charval0=6, charval1=1,charval2=0,charval3=7, sega=0011100,segb=1011110, segc=1110001, segd=0111000, @    100
sel = 3, charval0=6, charval1=1,charval2=0,charval3=7, sega=1011110,segb=0011100, segc=0111000, segd=1110001, @    110
sel = 0, charval0=0, charval1=1,charval2=2,charval3=3, sega=1111001,segb=1110111, segc=0111001, segd=0111000, @    120
sel = 0, charval0=1, charval1=1,charval2=2,charval3=3, sega=1111001,segb=1110111, segc=0111001, segd=0111001, @    130
sel = 0, charval0=2, charval1=1,charval2=2,charval3=3, sega=1111001,segb=1110111, segc=0111001, segd=1110111, @    140
sel = 0, charval0=3, charval1=1,charval2=2,charval3=3, sega=1111001,segb=1110111, segc=0111001, segd=1111001, @    150
sel = 0, charval0=3, charval1=7,charval2=2,charval3=3, sega=1111001,segb=1110111, segc=1110001, segd=1111001, @    160
sel = 0, charval0=3, charval1=6,charval2=2,charval3=3, sega=1111001,segb=1110111, segc=0111110, segd=1111001, @    170
sel = 0, charval0=3, charval1=5,charval2=2,charval3=3, sega=1111001,segb=1110111, segc=1110011, segd=1111001, @    180
$finish called from file "tb.v", line 78.
$finish at simulation time                 190
           V C S   S i m u l a t i o n   R e p o r t
Time: 190
CPU Time:      0.200 seconds;       Data structure size:   0.0Mb
Tue Sep 30 17:49:55 2025
[dthornburg@linux402604 Lab1]$
```

Conclusion

Overall, in this lab we created and demonstrated our character generator as well as proed we could write the characters in different orders due to the inclusion of the muxes in the circuit. We also feel that having good debugging statements in your testbench can be vital to help understand your output better. Sometimes, it's easier without the GUI.

Appendix

Place code and other data here.

CODE

```
module testbench();

reg [2:0] charval0;
reg [2:0] charval1;
reg [2:0] charval2;
reg [2:0] charval3;

wire [6:0] sega;
```

```
wire [6:0] segb;
wire [6:0] segc;
wire [6:0] segd;

reg [1:0] sel;

//added
wire [2:0] out_one;
wire [2:0] out_two;
wire [2:0] out_three;
wire [2:0] out_four;

// instantiate your module here:
four_one_mux one(charval3,charval2,charval1,charval0,sel,out_one);
four_one_mux two(charval2,charval3,charval0,charval1,sel,out_two);
four_one_mux three(charval1,charval0,charval3,charval2,sel,out_three);
four_one_mux four(charval0,charval1,charval2,charval3,sel,out_four);

Dylan_Char_Maker D_one(out_one, sega);
Dylan_Char_Maker D_two(out_two, segb);
Ryan_Char_Maker R_one(out_three, segc);
Ryan_Char_Maker R_two(out_four, segd);


initial begin
        $display("Start of the decoder test");
        $monitor("sel = %d, charval0=%d, charval1=%d,charval2=%d,charval3=%d,
sega=%b,segb=%b, segc=%b, segd=%b, @ %d", sel, charval0, charval1, charval2, charval3,
sega, segb, segc, segd, $time);
charval0 = 3'd0; charval1 = 3'd1; charval2 = 3'd2; charval3 = 3'd3;
sel = 0;
#10
sel = 3'd1;
#10
sel = 3'd2;
#10
sel = 3'd3;
#10
charval0 = 3'd2; charval1 = 3'd1; charval2 = 3'd2; charval3 = 3'd1;
sel = 0;
#10
sel = 3'd1;
#10
sel = 3'd2;
#10
sel = 3'd3;
#10
charval0 = 3'd6; charval1 = 3'd1; charval2 = 3'd0; charval3 = 3'd7;
sel = 0;
#10
sel = 3'd1;
#10
sel = 3'd2;
#10
sel = 3'd3;
```

```
#10
charval0 = 3'd0; charval1 = 3'd1; charval2 = 3'd2; charval3 = 3'd3;
sel = 0;
#10
charval0 = 3'd1;
#10
charval0 = 3'd2;
#10
charval0 = 3'd3;
#10
charval1 = 3'd7;
#10
charval1 = 3'd6;
#10
charval1 = 3'd5;
#10
$finish;

end

endmodule
```

```
module four_one_mux(
        input [2:0] sel0,
      input [2:0] sel1,
      input [2:0] sel2,
      input [2:0] sel3,
      input [1:0] mux_sel,
        output [2:0] out
);
        assign out = mux_sel[1] ? (mux_sel[0] ? sel3 : sel2) : (mux_sel[0] ? sel1 : sel0);
endmodule
```

```
module Dylan_Char_Maker (sel,segment);
        input  wire [2:0] sel;
        output reg [6:0] segment;
        always @* begin
                case (sel)
                3'b000: segment = 7'b1011100;
                3'b001: segment = 7'b0011100;
                3'b010: segment = 7'b1110111;
                3'b011: segment = 7'b1111001;
                3'b100: segment = 7'b1011000;
                3'b101: segment = 7'b1111100;
        3'b110: segment = 7'b1011110;
                3'b111: segment = 7'b1010100;
                default: segment = 7'b0000000;
        endcase
```

```
        end
endmodule
```

```
module Ryan_Char_Maker(sel,seg);
        input wire [2:0] sel;
        output reg [6:0] seg;
        always @* begin
                case (sel)
                3'b000: seg = 7'b0111000; //L
                3'b001: seg = 7'b0111001; //C
                3'b010: seg = 7'b1110111; //A
                3'b011: seg = 7'b1111001; //E
                3'b100: seg = 7'b1110110; //H
                3'b101: seg = 7'b1110011; //P
                3'b110: seg = 7'b0111110; //U
                3'b111: seg = 7'b1110001; //F
                default: seg = 7'b0000000;
                endcase
        end
endmodule
```